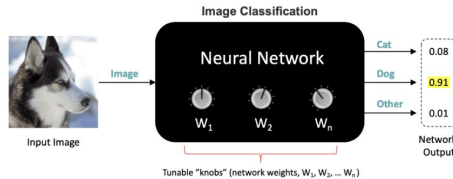
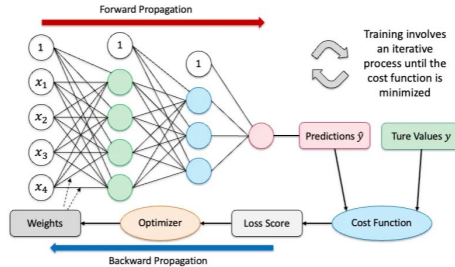


- Objectif : expliquer comment les réseaux de neurones sont entraînés
- Approche conceptuelle, sans entrer dans les détails d'architecture
- Thèmes abordés : données, fonction de perte, optimisation, boucle d'entraînement, inférence

Introduction



- Les données d'entraînement = exemples + étiquettes (labels)
- Pour la classification, les étiquettes doivent être numériques
- Encodage par entiers vs encodage One-Hot
- Exemple : classes (Chat, Chien, Autre)

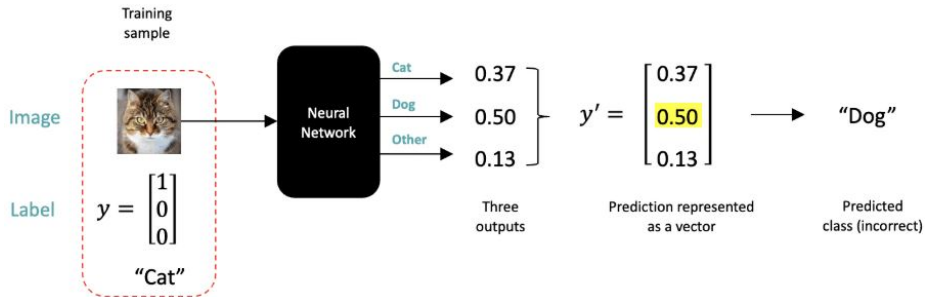
Exemple d'encodage des classes

- Deux façons courantes de représenter les classes :
- **Encodage en entiers :**
 - Chat : 0
 - Chien : 1
 - Autre : 2
- **Encodage One-Hot (pour 3 classes) :**
 - Chat : $[1 \ 0 \ 0]$
 - Chien : $[0 \ 1 \ 0]$
 - Autre : $[0 \ 0 \ 1]$
- L'encodage One-Hot évite d'introduire des relations artificielles entre classes

Introduction à la fonction de perte

- Considérons un seul exemple d'entraînement, composé :
 - de l'image d'entrée
 - de son étiquette de classe associée
- Pour chaque image d'entrée, le réseau produit une prédiction
- Cette prédiction est un vecteur de trois valeurs représentant les probabilités
- Chaque valeur correspond à une classe possible
- La classe prédite est celle ayant la probabilité la plus élevée
- Dans cet exemple, le réseau prédit incorrectement la classe « *Chien* »
- En effet, la deuxième sortie du réseau possède la probabilité la plus élevée
- L'entrée du réseau est uniquement l'image
- Les étiquettes réelles sont utilisées uniquement pour calculer la fonction de perte

Introduction à la fonction de perte



La fonction de perte (Loss Function)

- Tous les réseaux de neurones utilisent une fonction de perte
- Elle mesure l'erreur entre la sortie prédite et la vérité terrain (ground truth)
- Cette erreur sert à guider l'apprentissage du réseau
- Les poids sont mis à jour afin d'améliorer les prédictions futures
- **Somme des erreurs quadratiques (SSE)**
 - La SSE quantifie l'erreur pour un seul exemple d'entraînement
 - Elle est calculée en comparant chaque sortie prédite à la valeur réelle

$$SSE = (1 - 0.37)^2 + (0 - 0.50)^2 + (0 - 0.13)^2 = 0.6638$$

- En pratique, l'entraînement utilise plusieurs images à la fois
- On calcule alors l'erreur quadratique moyenne (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - y'_i)^2 = \text{moyenne des SSE}$$

- Le nombre d'images utilisées s'appelle la **taille du batch**
- Le traitement d'un batch correspond à une **itération**

Fonctions de perte en Deep Learning

Fonction	Type	Formule	Quand l'utiliser ?
MSE	Régression	$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Sorties continues. Sensible aux grandes erreurs.
MAE	Régression	$\text{MAE} = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Plus robuste aux valeurs aberrantes.
Binary Cross-Entropy	Classification binaire	$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$	Sortie sigmoïde, problème 0/1.
Categorical Cross-Entropy	Classification multi-classe	$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$	Classes exclusives avec softmax et encodage one-hot.
Sparse Categorical Cross-Entropy	Classification multi-classe	$L = -\log(\hat{y}_y)$	Labels entiers (0,1,2,...) sans one-hot.
Hinge Loss	Classification	$L = \max(0, 1 - y\hat{y})$	SVM, séparation par marge maximale.
Huber Loss	Régression	$L_\delta = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{si } y - \hat{y} \leq \delta \\ \delta(y - \hat{y} - \frac{1}{2}\delta) & \text{sinon} \end{cases}$	Compromis entre MSE et MAE.

Encodage des labels et type de problème

Élément	Classification	Régression
Nature du problème	Prédire une classe	Prédire une valeur numérique
Exemple de sortie	Chat / Chien / Oiseau	23.5°C, 150000€, 0.87
Format des labels	One-Hot : [0, 1, 0] Entiers : 0, 1, 2	Valeur réelle directe ($y \in \mathbb{R}$)
Activation en sortie	Softmax ou Sigmoid	Linéaire
Fonction de perte	Categorical Cross-Entropy Sparse Cross-Entropy	MSE / MAE / Huber
Nombre de neurones de sortie	C neurones (nb de classes)	1 neurone (ou k si multi-sortie)

Idée clé :

- One-hot et entiers sont deux manières de représenter des classes.
- En régression, il n'y a aucun encodage des labels.

1) Classification multi-classe (Images)

- Classes : Chat, Chien, Oiseau
- One-hot :
 - Chat $\rightarrow [1, 0, 0]$
 - Chien $\rightarrow [0, 1, 0]$
- Entiers :
 - Chat $\rightarrow 0$
 - Chien $\rightarrow 1$
- Sortie : Softmax + Cross-Entropy

2) Régression (Prix d'une maison)

- Label : 185000 €
- Pas d'encodage
- Sortie : neurone linéaire
- Loss : MSE

Fonction d'activation de sortie selon le problème

Type de problème	Activation en sortie	Remarques
Régression	Aucune (linéaire)	Sortie continue non bornée
Régression bornée	Sigmoïde / Tanh	Si la sortie doit être entre $[0,1]$ ou $[-1,1]$
Classification binaire	Sigmoïde	Produit une probabilité $P(y = 1)$
Classification multi-classe (exclusive)	Softmax	Probabilités normalisées : $\sum_i \hat{y}_i = 1$
Classification multi-label	Sigmoïde (par classe)	Plusieurs classes peuvent être vraies simultanément

Rappel mathématique :

- Sigmoïde : $\sigma(x) = \frac{1}{1 + e^{-x}}$
- Softmax : $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

Exemples concrets d'activation en sortie

1) Régression – Prédiction du prix d'une maison

- Sortie attendue : 185000 €
- Dernière couche : 1 neurone
- Activation : Linéaire
- Exemple de prédiction : $\hat{y} = 178450$

2) Classification binaire – Détection de spam

- Classes : Spam / Non-Spam
- Dernière couche : 1 neurone
- Activation : Sigmoidé
- Exemple de sortie : $\hat{y} = 0.87$ (87% spam)

3) Classification multi-classe – Reconnaissance d'images

- Classes : Chat, Chien, Oiseau
- Dernière couche : 3 neurones
- Activation : Softmax
- Exemple de sortie : $[0.70, 0.20, 0.10]$

- Étapes pour chaque batch :
 - ① Passage avant (forward pass)
 - ② Calcul de la perte
 - ③ Passage arrière (backpropagation)
 - ④ Mise à jour des poids
- Un passage complet sur les données = une époque
- Plusieurs époques améliorent l'apprentissage

Concepts clés de l'entraînement

- Batch size : nombre d'échantillons par mise à jour
- Époque : un passage complet sur le jeu de données
- Itération : traitement d'un batch
- Hyperparamètres : taux d'apprentissage, batch size, nombre d'époques

Inférence avec un modèle entraîné

- Après l'entraînement, le modèle est utilisé pour prédire
- Aucune étiquette n'est nécessaire à cette étape
- Les entrées sont converties en classes prédites

- Les données étiquetées sont essentielles pour l'apprentissage supervisé
- Le choix du type de problème (classification vs régression) détermine :
 - L'encodage des labels
 - L'activation de sortie
 - La fonction de perte
- La fonction de perte mesure l'erreur et guide l'optimisation
- La descente de gradient et la rétropropagation ajustent les poids
- L'entraînement se fait par batches, itérations et époques
- Les hyperparamètres influencent fortement la performance
- Après l'entraînement, le modèle est utilisé en phase d'inférence
- Un bon modèle nécessite : données de qualité, bonne architecture et réglage approprié

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to the document. If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.